

Софийски университет „Св. Климент Охридски“
Факултет по математика и информатика

Курсов Проект
Тема: Множество на Джулия (Julia
Set)

Дисциплина: Фрактали

Изготвил:

Факултетен №:

Курс:

Група:

Специалност:

град София

Съдържание

1. Теоретична част.....	3
1.1. Гастон Джулия (Gaston Maurice Julia).....	3
1.2. Множество на Джулия.....	3
1.2.1. Определение и дефиниция.....	3
1.2.2. Основни свойства.....	4
1.2.2.1. Инвариантност:.....	4
1.2.2.2. Свързаност:.....	4
1.2.2.3. Фрактална размерност:.....	4
1.2.2.4. Самоподобие.....	4
1.2.2.5. Некалкулируемост.....	4
1.2.3. Приложения на множествата на Джулия.....	4
1.2.4. Визуализация на множеството на Джулия.....	5
1.3. Използвана литература.....	5
2. Практическа част.....	6
2.1. Инструкции за пускане на програмата и очаквани резултати.....	6
2.2. Описание на софтуерната реализация.....	7
2.2.1. Архитектура и графичен интерфейс.....	7
2.2.2. Преобразуване на координатите (Метод map).....	7
2.2.3. Изчисляване на фракталната динамика (Метод computeIterations).....	7
2.2.4. Нелинейно синусоидално оцветяване (Метод getColor).....	8
2.3. Програмен код.....	8

1. Теоретична част

1.1. Гастон Джулия (Gaston Maurice Julia)

Gaston Maurice Julia (3 февруари 1893 - 19 март 1978) е френски математик, известен с пионерската си работа в областта на комплексната динамика. Той е един от основоположниците на изследването на итеративните процеси в комплексната равнина, които по-късно стават основа за фракталите, носещи неговото име - множества на Джулия.

Роден е в семейство на търговец в Сиди Бел Абес, Френски Алжир. По време на Първата световна война служи във френската армия и губи носа си, след което до края на живота си носи кожена протеза. Този инцидент не му попречва да се отдаде на математиката.

През 1917 г., докато е в болница, Джулия започва да работи върху итерацията на рационални функции. През 1918 г. публикува своя 199-страничен мемоар в *Journal de Mathématiques Pures et Appliquées*, който му донеся Голямата награда на Френската академия на науките. В него той описва детайлно това, което днес наричаме множество на Джулия.

Въпреки че работата му остава относително незабелязана десетилетия наред, тя придобива огромна популярност през 80-те години на XX век, благодарение на Беноа Манделброт и компютърната визуализация. Джулия умира през 1978 г., без да види широкото признание на своите открития.

1.2. Множество на Джулия

1.2.1. Определение и дефиниция

Множеството на Джулия се дефинира чрез изучаване на поведението на последователности от комплексни числа, генерирани от итерацията на комплексна функция. Най-често изследваният случай е квадратното изображение:

$$f_c(z) = z^2 + c$$

Където z е променлива в комплексна равнина ($z = x + i*y$), а c е фиксиран комплексен параметър ($c = c_{re} + i c_{im}$).

За всяка начална точка z_0 в комплексните числа дефинираме итерационната редица:

$$z_1 = f_c(z_0), \quad z_2 = f_c(z_1), \quad \dots, \quad z_{n+1} = f_c(z_n)$$

В зависимост от избора на z_0 , тази редица може или да клони към безкрайност, или да остане ограничена в рамките на определен диск в комплексната равнина.

- Множество на Фату: Множеството от начални точки z_0 , за които динамиката е стабилна (точките се държат предвидимо).
- Множество на Джулия: Границата между точките, които клонят към безкрайност, и тези, които остават ограничени. В това множество динамиката е хаотична.

1.2.2. Основни свойства

1.2.2.1. Инвариантност:

$$f(J_c) = f^{-1}(J_c) = J_c$$

1.2.2.2. Свързаност:

- Ако s принадлежи на множеството на Манделброт, то J_c е свързано.
- В противен случай J_c е канторово множество (напълно несвързано, „прах“).

1.2.2.3. Фрактална размерност:

Размерността на J_c е между 1 и 2, като зависи от c .

1.2.2.4. Самоподобие

J_c е инвариантно относно fc и обратната ѝ, което води до самоподобна структура.

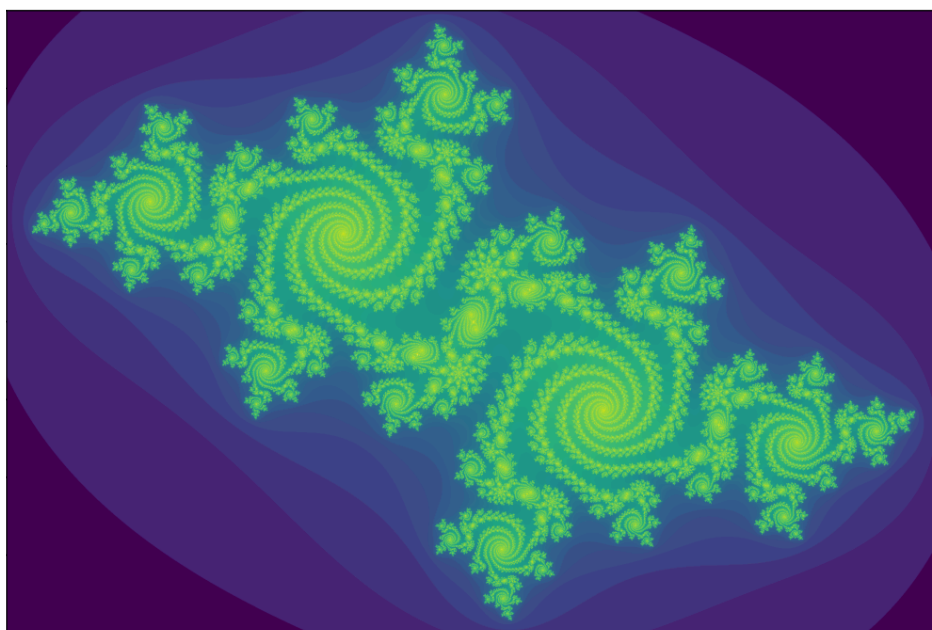
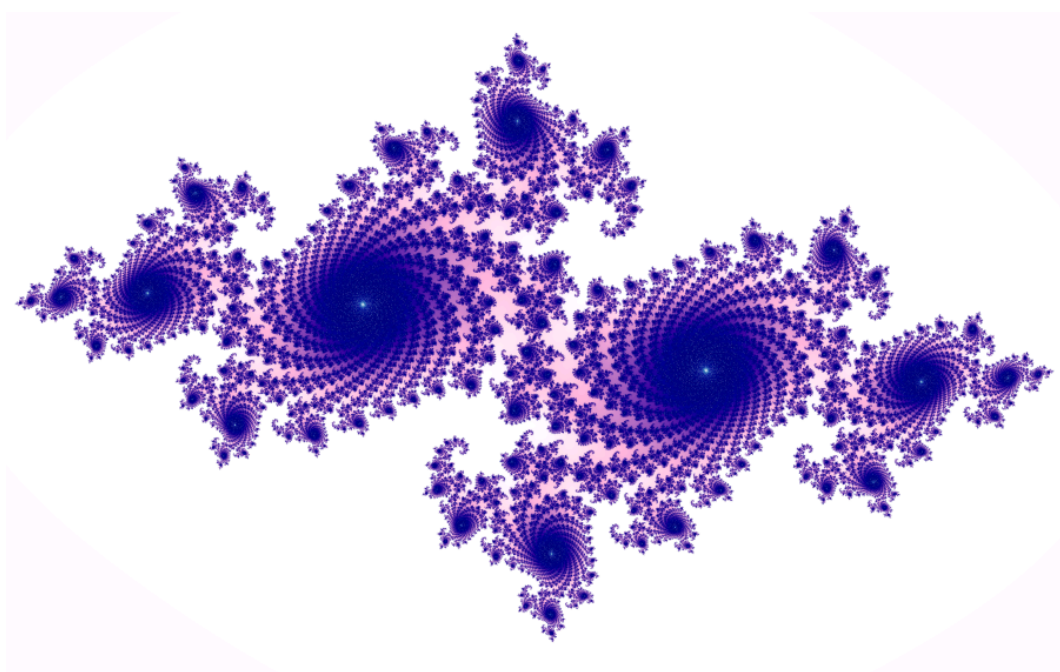
1.2.2.5. Некалкулируемост

За повечето стойности на c множеството е изключително сложно и няма просто аналитично описание - визуализира се числено.

1.2.3. Приложения на множествата на Джулия

- Комплексна динамика - изучаване на поведението на итерациите.
- Компютърна графика - генериране на художествени фрактални изображения.
- Криптография - използване на фрактални функции за генериране на псевдослучайни числа.
- Моделиране в природата - форми на облаци, брегови линии, дървета.
- Теория на информационните системи - анализ на хаотични системи.

1.2.4. Визуализация на множеството на Джулия



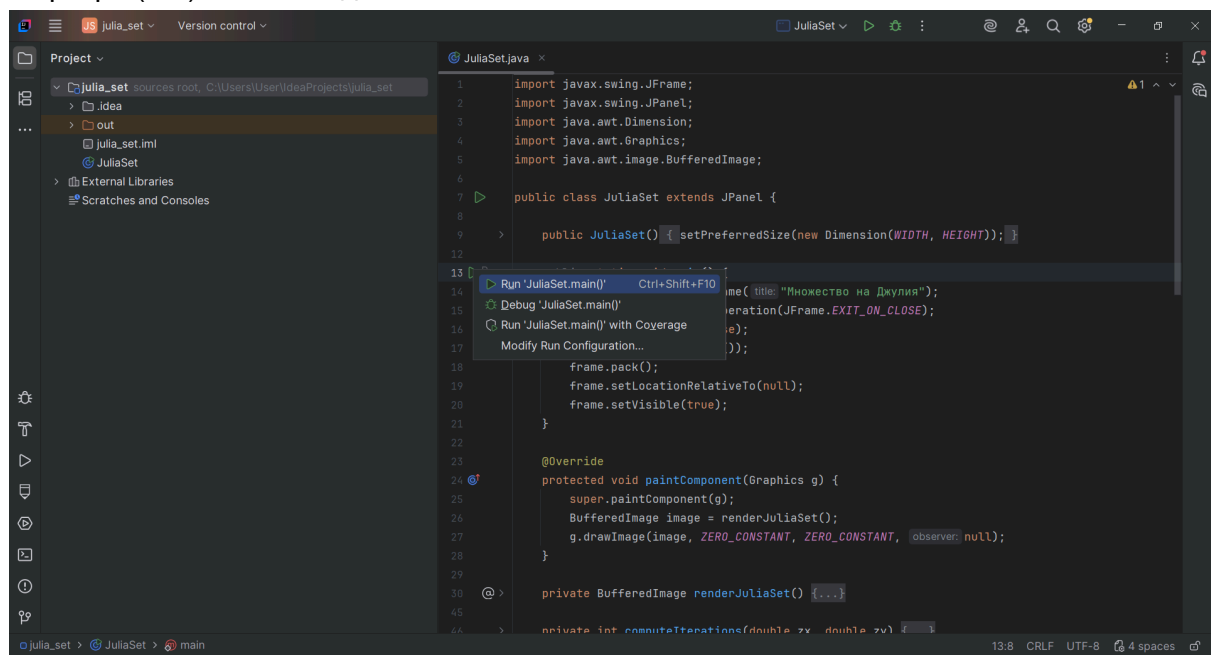
1.3. Използвана литература

- Kenneth Falconer – "Fractal Geometry: Mathematical Foundations and Applications", Wiley, 1990.
- Wikipedia - "Julia set" (енгл. и бълг. версии)

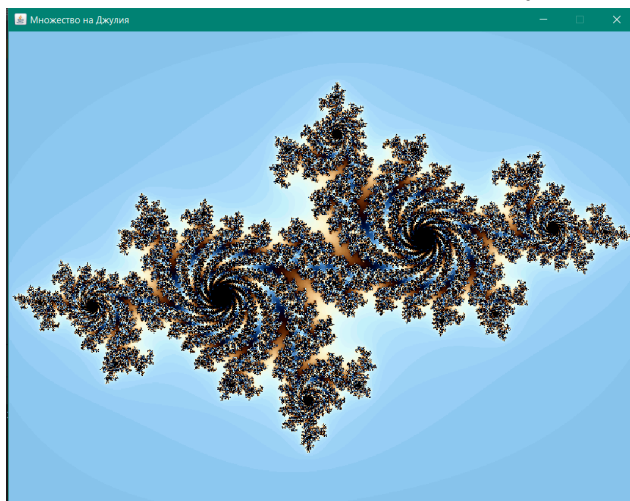
2. Практическа част

2.1. Инструкции за пускане на програмата и очаквани резултати

За разработката на “практическата” част от курсовия проект съм използвала IntelliJIde. На диска, предаден с доклада, е поставен JuliaSet.java файл. За да бъде успешно пуснат файлът, се очаква той да бъде импортиран и изпълнен в среда за разработка (IDE), поддържаща Java (например IntelliJ IDEA, Eclipse или NetBeans), с предварително конфигуриран Java Development Kit (JDK) версия 8 или по-нова. При успешно импортиране на файлът в среда за разработка, трябва да се стартира (run) main методът.



При успешно стартиране на файлът трябва да изскочи прозорец с изображението на множеството на Джулия.



2.2. Описание на софтуерната реализация

Програмата е реализирана на езика Java, версия 25, като използва обектно-ориентиран подход и стандартната графична библиотека `javax.swing`. Структурата е проектирана така, че да спазва предварително зададения правила за clean code (чистия код).

2.2.1. Архитектура и графичен интерфейс

Класът `JuliaSet` наследява `JPanel` (компонент за рисуване в Java Swing).

- **Метод `main`:** Инициализира главния прозорец (`JFrame`), задава неговите свойства (размери, невъзможност за оразмеряване от потребителя с цел запазване на аспектното съотношение) и го позиционира в центъра на екрана чрез `setLocationRelativeTo(null)`.
- **Метод `paintComponent`:** Пренаписва стандартния метод за изчертаване. Той извиква изчисления фрактал под формата на `BufferedImage` и го рисува на екрана наведнъж чрез `g.drawImage(image, ZERO_CONSTANT, ZERO_CONSTANT, null)`. Този подход предотвратява ефекта на трептене (flickering) при рендериране.

2.2.2. Преобразуване на координатите (Метод `map`)

Компютърният екран работи в пикселни координати (x , y), където горният ляв ъгъл е (0,0), а долният десен е (WIDTH, HEIGHT). Математическата итерация обаче се извършва в комплексната равнина в предварително дефинирани непрекъснати граници: x в $[-1.5, 1.5]$ и y в $[-1.125, 1.125]$.

За целта е реализирана помощната функция ***map***. Тя извършва линейна интерполация, преобразувайки дискретния пикселен индекс в реално число от комплексната равнина:

```
private double map(double value, double fromHigh, double toLow,
double toHigh) {
    return toLow + (value - (double) JuliaSet.ZERO_CONSTANT) *
(toHigh - toLow) / (fromHigh - (double) JuliaSet.ZERO_CONSTANT);
}
```

2.2.3. Изчисляване на фракталната динамика (Метод `computeIterations`)

Методът `computeIterations(double zx, double zy)` имплементира алгоритъма Escape Time Algorithm за комплексното изображение

$$f_c(z) = z^2 + c$$

2.2.4. Нелинейно синусоидално оцветяване (Метод getColor)

За да се постигне високо визуално качество и плавни преходи без резки цветови граници (Color Banding), методът getColor използва тригонометрични вълнови функции:

```
double t = (double) iterations / MAX_ITERATIONS;  
int r = (int) (Math.sin(COLOR_CYCLE_FREQUENCY * t +  
    PHASE_RED) * COLOR_AMPLITUDE + COLOR_MIDPOINT);
```

- Стойността t се нормализира в интервала [0, 1] на база реално извършените итерации преди “бягството”.
- Функцията Math.sin връща стойност между -1 и 1. За да се превърне това в валиден RGB компонент (цяло число от 0 до 255), резултатът се умножава по амплитуда COLOR_AMPLITUDE (127.5) и се измества с център COLOR_MIDPOINT (127.5).
- Чрез промяна на фазовите отмествания (PHASE_RED, PHASE_GREEN, PHASE_BLUE) се контролира кои цветови канали да доминират, създавайки специфичната мека и хармонична палитра.
- Битови операции: Накрая, трите 8-битови променливи за каналите се пакетират в едно общо 32-битово цяло число чрез побитово изместване наляво (<<) и побитово “ИЛИ” (|), което спестява памет и ускорява работата с пикселите на BufferedImage.

2.3. Програмен код

```
import javax.swing.JFrame;  
import javax.swing.JPanel;  
import java.awt.Dimension;  
import java.awt.Graphics;  
import java.awt.image.BufferedImage;  
  
public class JuliaSet extends JPanel {  
  
    public JuliaSet() {  
        setPreferredSize(new Dimension(WIDTH, HEIGHT));  
    }  
  
    public static void main() {  
        JFrame frame = new JFrame("Множество на Джулия");  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        frame.setResizable(false);
```



```

        frame.add(new JuliaSet());
        frame.pack();
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        BufferedImage image = renderJuliaSet();
        g.drawImage(image, ZERO_CONSTANT, ZERO_CONSTANT, null);
    }

    private BufferedImage renderJuliaSet() {
        BufferedImage image = new BufferedImage(WIDTH, HEIGHT,
        BufferedImage.TYPE_INT_RGB);

        for (int px = 0; px < WIDTH; px++) {
            for (int py = 0; py < HEIGHT; py++) {
                double zx = map(px, WIDTH, X_MIN, X_MAX);
                double zy = map(py, HEIGHT, Y_MIN, Y_MAX);

                int iterations = computeIterations(zx, zy);
                int color = getColor(iterations);
                image.setRGB(px, py, color);
            }
        }
        return image;
    }

    private int computeIterations(double zx, double zy) {
        int iterator = 0;
        while (zx * zx + zy * zy < ESCAPE_RADIUS_SQ && iterator <
        MAX_ITERATIONS) {
            double tmp = zx * zx - zy * zy + C_REAL;
            zy = COMPLEX_2AB_FACTOR * zx * zy + C_IMAG;
            zx = tmp;
            iterator++;
        }
        return iterator;
    }

    private int getColor(int iterations) {
        if (iterations == MAX_ITERATIONS) {
            return COLOR_INSIDE_SET;
        }

        double t = (double) iterations / MAX_ITERATIONS;

```

```

        int r = (int) (Math.sin(COLOR_CYCLE_FREQUENCY * t + PHASE_RED) *
COLOR_AMPLITUDE + COLOR_MIDPOINT);
        int g = (int) (Math.sin(COLOR_CYCLE_FREQUENCY * t + PHASE_GREEN) *
COLOR_AMPLITUDE + COLOR_MIDPOINT);
        int b = (int) (Math.sin(COLOR_CYCLE_FREQUENCY * t + PHASE_BLUE) *
COLOR_AMPLITUDE + COLOR_MIDPOINT);

        return (r << RED_BIT_SHIFT) | (g << GREEN_BIT_SHIFT) | b;
    }

    private double map(double value, double fromHigh, double toLow, double toHigh) {
        return toLow + (value - (double) JuliaSet.ZERO_CONSTANT) * (toHigh - toLow)
/ (fromHigh -
        (double) JuliaSet.ZERO_CONSTANT);
    }

    // window dimensions
    private static final int WIDTH = 800;
    private static final int HEIGHT = 600;

    // fractal params
    private static final int MAX_ITERATIONS = 300;
    private static final double C_REAL = -0.7;
    private static final double C_IMAG = 0.27015;

    // limits of complex plane
    private static final double X_MIN = -1.5;
    private static final double X_MAX = 1.5;
    private static final double Y_MIN = -1.125;
    private static final double Y_MAX = 1.125;

    // mathematical constants
    private static final double ESCAPE_RADIUS_SQ = 4.0;
    private static final double COMPLEX_2AB_FACTOR = 2.0;
    private static final int ZERO_CONSTANT = 0;

    // color constants
    private static final int COLOR_INSIDE_SET = 0x000000;
    private static final double COLOR_CYCLE_FREQUENCY = 18.0;

    // phase offsets for colors
    private static final double PHASE_RED = 0.0;
    private static final double PHASE_GREEN = 0.5;
    private static final double PHASE_BLUE = 1.0;

    // Amplitude and center for scaling in the range [0, 255]

```

```
private static final double COLOR_MIDPOINT = 127.5;
private static final double COLOR_AMPLITUDE = 127.5;

// Bit shifts for the RGB channels
private static final int RED_BIT_SHIFT = 16;
private static final int GREEN_BIT_SHIFT = 8;

}
```